编译系统原理预备工作实验报告
了解你的编译器：编译流程与目标代码实现

谭锦睿（2412771）　王帅（2413995）

目录

1 引言 **3**
2 实验任务与小组分工 **3**
2.1 实验任务 . . . 3
2.2 小组分工 . . . 4
3 C 程序编译流程观察 **4**
3.1 实验程序与总体流程 . . . 4
3.2 预处理、词法与语法结构 . . . 4
3.3 LLVM IR、汇编与链接 . . . 5
3.4 优化前后的代码差异 . . . 6
3.5 语言处理系统各阶段职责总结 . . . 7
4 SysY 测试程序设计 **7**
4.1 程序功能与设计思路 . . . 7
4.2 核心计算过程与语言特性覆盖 . . . 8
4.3 测试用例与预期结果 . . . 9
5 LLVM IR 实现与验证 **10**
5.1 总体结构与基本块设计 . . . 10
5.2 局部变量与数组寻址 . . . 10
5.3 控制流与整数运算 . . . 11
5.4 函数调用与 SysY 运行时接口 . . . 13
5.5 编译、链接与执行验证 . . . 13
6 RISC-V 汇编实现与验证 **15**
6.1 运行环境与总体实现 . . . 15
6.2 栈帧与数组寻址 . . . 15
6.3 控制流与整数运算 . . . 16
6.4 函数调用与运行时库 . . . 17
6.5 汇编、链接与执行验证 . . . 17
7 LLVM IR 与 RISC-V 实现结果比较 **18**
8 进阶探索：MLIR 与 AscendNPU IR **19**
8.1 从 LLVM IR 到多层中间表示 . . . 19
8.2 AscendNPU IR 官方 VecAdd 示例分析 . . . 19
8.3 多级 IR 与逐层 Lowering . . . 20
8.4 与 LLVM IR 的比较 . . . 20

9 总结 **21**
参考文献 **21**

摘要

本实验围绕编译器从高级语言源程序到目标程序的处理过程展开。首先以循环实现的阶乘 C 程序为例，利用 GCC 和 Clang/LLVM 观察预处理、词法分析、抽象语法树、LLVM IR、汇编、目标文件和链接等阶段的输出，并比较不同优化级别对生成代码的影响。在此基础上，设计了一段覆盖常量、数组、算术运算、条件分支、循环、函数调用和运行时输入输出的 SysY 程序，分别进行 LLVM IR 和 RISC-V 目标表示实现。RISC-V 部分进一步与 SysY 运行时库进行静态链接，并通过 QEMU 对普通输入和边界输入进行验证。进阶部分结合 AscendNPU IR 的 VecAdd 示例，对 MLIR 多层 Dialect 与渐进式 lowering 的基本思想进行了文档调研和分析。通过上述实验，加深了对编译器前端、中间表示、目标代码和运行时环境之间关系的理解。

关键词：编译器；LLVM IR；SysY；RISC-V；MLIR

1 引言

编译系统的作用并不仅是将高级语言源程序一次性转换为机器代码，而是通过预处理、词法分析、语法分析、语义分析、中间代码生成与优化、目标代码生成、汇编和链接等多个阶段，逐步降低程序的抽象层次，最终形成能够在目标平台运行的可执行程序。

LLVM 将中间表示设计为相对独立于源语言和目标体系结构的 LLVM IR，使前端分析、通用优化与后端代码生成能够相对解耦。另一方面，面向 AI 加速器的编译系统还需要表达张量操作、专用存储层次、数据搬运以及硬件计算单元等信息，因此 MLIR 进一步采用多层中间表示和 Dialect 机制，允许程序在不同抽象层次之间逐级 lowering。

本实验首先以一个简单 C 程序为例，实际观察 GCC 和 Clang/LLVM 语言处理系统从源程序到可执行文件的完整过程，并研究不同编译阶段输出与源程序之间的关系。在此基础上，两位成员共同设计统一的 SysY 测试程序和测试用例，随后分别完成 LLVM IR 与 RISC-V 汇编两种等价低层实现，并通过 SysY 运行时库和实际执行验证其正确性。

此外，为进一步理解现代 AI 编译器中端的设计思想，本实验结合 AscendNPU IR 官方 VecAdd 示例，对 MLIR 的多级 Dialect、存储空间表达和逐层 lowering 过程进行文档调研与分析。

2 实验任务与小组分工

2.1 实验任务

本次预备工作围绕“了解编译器”展开，目标并不是只完成一次程序编译，而是通过实际工具观察高级语言程序在编译过程中经历的不同表示形式，并理解这些表示之间的关系。

基础部分首先选择一个规模适中的 C 程序，利用 GCC 和 Clang/LLVM 提供的命令行选项，依次观察预处理结果、词法单元、抽象语法树、LLVM IR、汇编程序、目标文件和最终可执行文件。通过这些中间产物，可以将课堂中的编译流程具体落实到实际文件和指令中。

在此基础上，进一步设计具有代表性的 SysY 程序，使其覆盖数值运算、赋值、条件分支、循环、函数调用以及数组等语言结构，并分别手工实现等价 LLVM IR 和 RISC-V 汇编代码。实现后与课程提供的 SysY 运行时库进行链接，通过实际执行验证不同低层表示是否保持与源程序一致的语义。

进阶部分结合 AscendNPU IR 的 VecAdd 示例，对 MLIR 的 Dialect、多层中间表示以及渐进式 lowering 思想进行调研，从而将实验范围由传统通用编译器扩展到面向 AI 加速器的编译流程。

实验主要在 Windows + WSL2 Ubuntu 环境下完成。第一部分使用 GCC 13.3.0 和 LLVM 18；RISC-V 部分使用 GCC 15.1.0、GNU Binutils 2.45 和 qemu-riscv64 8.2.2。

2.2 小组分工

第一部分由两位成员共同完成，包括实验程序设计、编译流程命令执行、中间结果观察以及优化前后代码比较。第二部分以同一 SysY 测试程序作为语义基准，在保证最终输入输出一致的基础上分别完成 LLVM IR 和 RISC-V 两种实现。

表 1: 小组成员分工

姓名	学号	主要工作
谭锦睿	2412771	负责 RISC-V 汇编实现，包括栈帧设计、数组地址计算、条件与循环控制流翻译、32 位整数运算、函数调用、SysY RISC-V 运行时库链接及 QEMU 执行验证；参与第一部分实验和进阶内容整理。
王帅	2413995	负责 SysY 程序的 LLVM IR 手工实现，包括基本块组织、数组寻址、控制流翻译、运行时函数声明与链接以及执行验证；与谭锦睿共同完成第一部分实验。

3 C 程序编译流程观察

3.1 实验程序与总体流程

实验选取循环实现的阶乘程序作为贯穿各阶段的示例。程序定义 `factorial(int n)` 函数，通过循环不断更新累乘结果，主函数读取整数后调用该函数并输出结果。该程序虽然规模较小，但同时包含函数定义、局部变量、循环、整数运算以及标准库函数调用，能够较完整地观察编译过程中不同阶段的变化。

输入 5 时程序输出 120，说明后续观察到的各类中间结果均对应一个功能正确的源程序。

```
tjr@tjry7000p:~/compiler-lab/prework1/part1$ cd ~/compiler-lab/prework1/part1
gcc -Wall -Wextra -O0 main.c -o main
printf "5\n" | ./main
Input n: factorial = 120
```

图 1: 阶乘程序运行结果

整个处理过程可以概括为：

$$\text{main.c} \rightarrow \text{main.i} \rightarrow \text{Token / AST} \rightarrow \text{LLVM IR} \rightarrow \text{Assembly} \rightarrow \text{Object} \rightarrow \text{Executable}.$$

这些阶段并不是简单的文件格式变化，而是在不断降低程序的抽象层次。源程序强调程序员能够理解的语义结构，AST 强调语法关系，LLVM IR 开始显式体现基本块和数据访问，汇编程序则已经直接面向具体目标体系结构。

3.2 预处理、词法与语法结构

首先使用 GCC 的预处理选项：

```
gcc -E main.c -o main.i
```

源文件 main.c 只有 33 行，而生成的 main.i 达到 847 行。主要原因是 stdio.h 等标准头文件中的声明被展开，同时源程序中定义的宏 START_VALUE 已经被替换为常量 1。

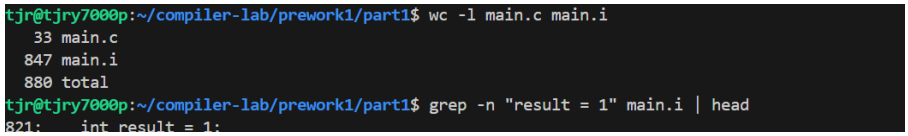
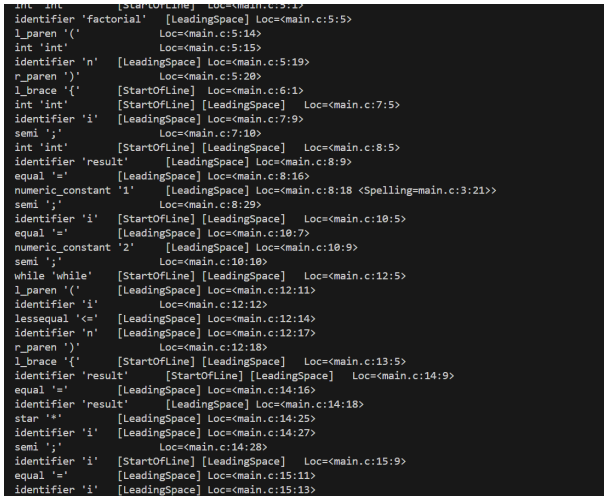


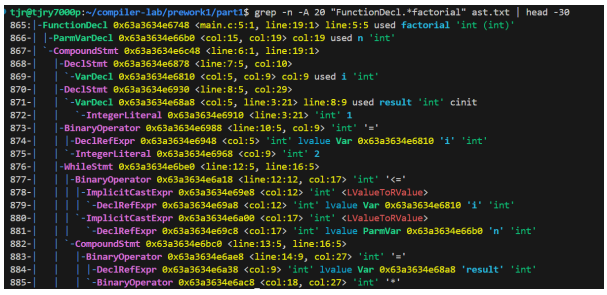
图 2: 预处理前后的文件规模及宏展开结果

因此，预处理阶段仍属于源语言层面的处理。它不会直接生成机器代码，而是完成宏替换、头文件展开等工作，为后续真正的编译阶段准备完整输入。

进一步使用 Clang 输出 Token 和 AST。Token 输出中可以看到 int、while 等关键字，factorial、result 等标识符，以及括号、运算符和数值常量。AST 则进一步将这些线性的词法单元组织为树形语法结构。例如，factorial 对应 FunctionDecl 节点，循环对应 WhileStmt，表达式中的乘法和加法则对应 BinaryOperator。



(a) Token 输出



(b) AST 片段

图 3: 词法单元与抽象语法树

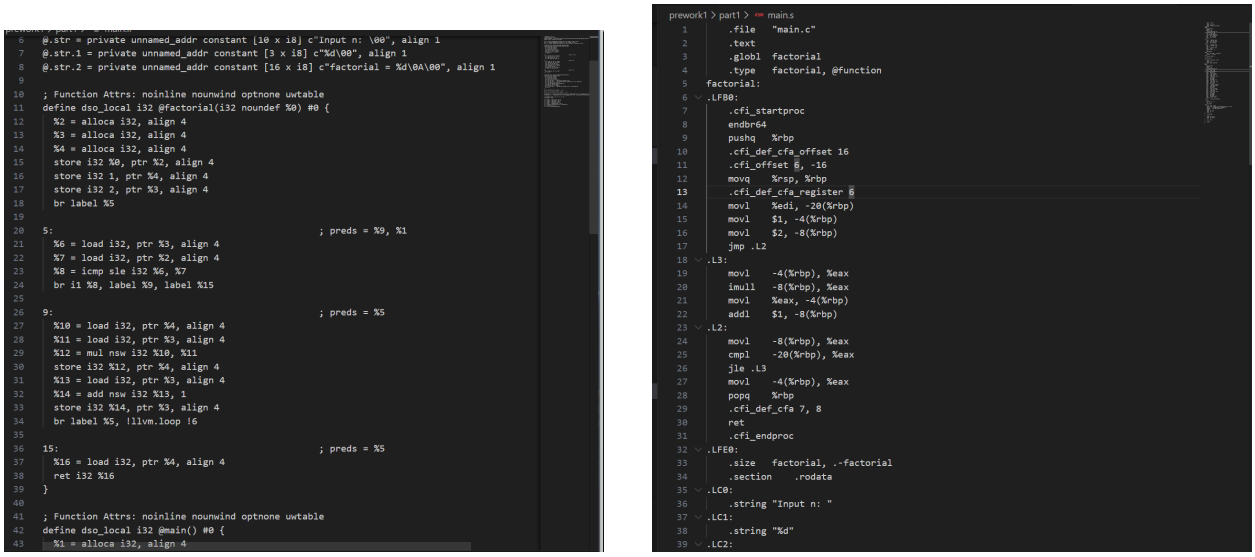
这一过程体现了词法分析和语法分析的区别：前者回答“源程序由哪些词法单元组成”，后者进一步回答“这些词法单元按照什么语法结构组合”。

3.3 LLVM IR、汇编与链接

使用

```
clang -S -emit-llvm -O0 main.c -o main.ll
```

生成 LLVM IR 后，可以观察到源程序中的局部变量通过 alloca 分配空间，通过 load 和 store 进行读写；循环条件使用 icmp 完成判断，并由 br 在不同基本块之间转移；乘法和循环变量递增则分别使用 mul 和 add。



(a) LLVM IR (b) x86-64 汇编

图 4: LLVM IR 与目标汇编代码

LLVM IR 已经明显低于 C 语言的抽象层次，但仍不依赖具体机器寄存器。继续生成 x86-64 汇编后，局部变量开始表现为寄存器或者栈偏移，条件和循环进一步转换为比较与跳转指令。

目标文件阶段通过 `nm` 可以观察到 `factorial` 和 `main` 已在当前翻译单元中定义，而 `printf`、`scanf` 等符号仍需要外部库提供。经过链接后，最终形成 ELF 可执行文件。

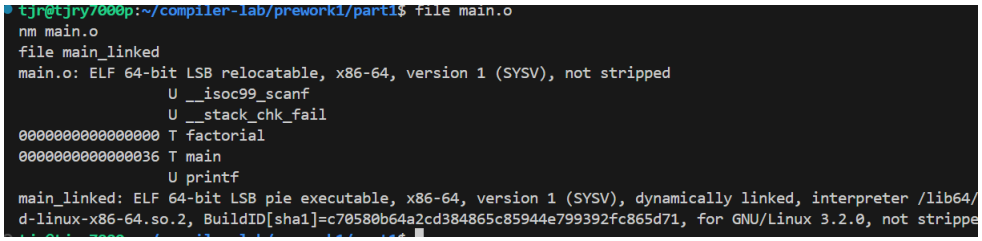


图 5: 目标文件符号与链接结果

这一现象说明，“编译完成”和“程序已经可以独立执行”并不是同一个概念。编译器可以先生成包含未解析外部符号的目标文件，再由链接器负责将目标文件和所需库组合起来。

3.4 优化前后的代码差异

实验还分别生成了 `-O0` 与 `-O2` 版本的汇编代码。未优化情况下，`i` 和 `result` 等局部变量频繁通过栈访问，控制流也基本保持源程序的形式。

例如 `-O0` 中循环主体包含：

```
movl -4(%rbp), %eax
imull -8(%rbp), %eax
movl %eax, -4(%rbp)
addl $1, -8(%rbp)
```

而 `-O2` 中对应循环被重新组织为：

```
imull %eax, %edx
```

```
leal 1(%rax), %ecx
addl $2, %eax
imull %ecx, %edx
```

可以看到优化后更多中间值直接保存在寄存器中，而且一次循环可以完成两个相邻乘数的累乘，减少循环控制次数。

此外，-O2 版本的 main 中原有的 call factorial 消失，阶乘计算逻辑被直接展开到调用位置，体现出函数内联优化。

因此，编译器优化的本质并不是简单缩短汇编文件，而是在保证程序外部可观察语义不变的前提下重新组织数据流、控制流和函数调用，从而降低访存、分支和调用等开销。

3.5 语言处理系统各阶段职责总结

结合上述实验，可以进一步明确语言处理系统各组成部分的职责。

表 2: 语言处理系统主要组成部分及作用

组成部分	主要作用	本实验中的观察结果
预处理器	处理头文件展开、宏替换和条件编译等源代码级操作，为后续编译阶段生成完整输入	main.i
编译器	依次完成词法分析、语法分析、语义分析、中间代码生成与优化，并进一步进行目标代码生成。其中词法分析识别 Token，语法分析建立 AST，语义分析检查类型、声明和作用域等约束	Token、AST、LLVM IR、汇编代码
汇编器	将目标汇编指令编码为机器指令，生成可重定位目标文件，同时保留符号表和必要的重定位信息	main.o
链接器	解析不同目标文件和库之间的符号引用，完成重定位，将程序与所需库组合为最终可执行文件	ELF 可执行程序

因此，从源程序到可执行程序并不是一次简单的转换，而是预处理、编译、汇编和链接多个阶段协同完成的过程。

4 SysY 测试程序设计

4.1 程序功能与设计思路

第二部分实验需要分别手工实现 LLVM IR 和 RISC-V 汇编。为了使两种低层表示能够建立在同一语义基准上，本实验首先共同设计了一段规模适中、同时包含多种 SysY 语言结构的测试程序。

程序由 main 和 calc 两个函数组成。其中，main 负责读取输入、执行上下界检查、调用计算函数并完成最终结果输出；calc 则负责主要循环计算过程。

主函数首先通过

```
n = getint();
```

读取一个整数。为了保证后续访问

```
int values[8];
```

时不会发生数组越界，程序将输入限制在区间 $[1, 8]$：

```
if (n < 1)
{
    n = 1;
}
else
{
    if (n > LIMIT)
    {
        n = LIMIT;
    }
}
```

其中常量

```
const int LIMIT = 8;
```

与数组长度保持一致。

完成输入修正后，主函数调用

```
result = calc(n);
```

得到主要计算结果，并继续执行整数除法和取模：

```
quotient = n / 2;
remainder = n % 2;
```

最后将 calc 的返回值、商、余数以及常量 BASE=3 相加，并通过 putint 和 putch 输出结果。

因此，该程序既包含较典型的数值计算，又能够覆盖数组、循环、条件分支、函数调用和运行时接口等后续低层实现中需要重点处理的结构。

4.2 核心计算过程与语言特性覆盖

calc 函数是程序的核心计算部分。函数内部定义长度为 8 的整型数组：

```
int values[8];
```

并使用循环依次计算数组前 n 个元素：

```
while (i < n)
{
    if ((i + 1) % 2 == 0)
    {
        values[i] = (i + 1) * (i + 1);
    }
    else
    {
        values[i] = -(i + 1);
    }

    result = result + values[i];
    i = i + 1;
}
```

当 $i+1$ 为偶数时，将其平方写入数组；当 $i+1$ 为奇数时，则将其相反数写入数组。每次完成数组赋值后，将当前元素累加到 result 中。

例如，当

$$n = 5$$

时，数组前五项依次为

$$[-1,\ 4,\ -3,\ 16,\ -5].$$

因此：

$$\texttt{calc}(5) = -1 + 4 - 3 + 16 - 5 = 11.$$

主函数继续计算：

$$5/2 = 2, \qquad 5 \bmod 2 = 1,$$

并加入：

$$\texttt{BASE} = 3,$$

所以最终得到：

$$11 + 2 + 1 + 3 = 17.$$

这一计算过程既可以人工验证，也可以作为后续 LLVM IR 和 RISC-V 两种实现的语义检查基准。

程序覆盖的主要 SysY 语言结构如表所示。

表 3: SysY 测试程序覆盖的主要语言特性

类别	使用内容
数据定义	常量、局部整型变量、一维整型数组
算术运算	加法、减法、乘法、整数除法、取模和一元负号
关系运算	小于、大于和等于比较
控制结构	嵌套 `if/else` 和 `while` 循环
函数机制	函数定义、参数传递、返回值和函数调用
运行时接口	`getint`、`putint`、`putch`

这样的程序规模不会使手工翻译过于复杂，同时又能够较完整地观察高级语言结构在 LLVM IR 和 RISC-V 汇编中的不同实现方式。

4.3 测试用例与预期结果

为了验证两种实现的正确性，本实验为 LLVM IR 和 RISC-V 使用完全相同的测试输入。

除了普通输入外，还专门设置了超出上下界的输入，用于检查边界修正逻辑。

表 4: 统一测试用例与预期结果

输入	实际参与计算的 n	预期输出	测试目的
0	1	3	验证下界修正
1	1	3	验证下界边界
4	4	21	验证普通偶数输入
5	5	17	验证普通奇数输入
8	8	111	验证上界边界
10	8	111	验证上界修正

其中输入 0 会在主函数中被修正为 1；输入 10 会被限制为 8。因此，这两组用例可以专门检查条件分支是否按照源程序语义正确执行。

其余输入则主要用于验证数组寻址、循环、乘法、除法、取模以及函数调用等实现。

通过统一测试程序和统一输入，可以使后续 LLVM IR 与 RISC-V 的验证结果具有直接可比性。

5 LLVM IR 实现与验证

5.1 总体结构与基本块设计

主要负责人：王帅（2413995）

LLVM IR 部分以第 3 章中的 SysY 程序作为语义基准，手工完成 `calc` 和 `main` 两个函数的中间表示。

实现采用接近未优化编译结果的直接翻译方式：局部变量首先使用 `alloca` 在函数栈帧中分配空间，随后通过 `load` 和 `store` 完成读写。

与 SysY 中按照语句顺序组织程序不同，LLVM IR 中控制流以基本块为单位组织。每个基本块以标签开始，并以 `br` 或 `ret` 等终结指令结束。

以 `calc` 为例，主要基本块包括：

表 5: `calc` 函数基本块及作用

基本块	主要作用
`entry`	分配局部变量并完成初值设置
`loop.cond`	读取 i 和 n，判断是否继续执行循环
`loop.body`	计算 $i+1$、奇偶性和数组元素地址
`if.then`	处理偶数情况，将平方写入数组
`if.else`	处理奇数情况，将负值写入数组
`if.end`	读取数组元素、更新累加结果和循环变量
`loop.end`	读取最终结果并返回

因此，源程序中的结构化控制语句，在 LLVM IR 中被显式转换为多个基本块之间的有向控制流。

5.2 局部变量与数组寻址

SysY 中的局部整型变量统一使用 LLVM IR 的 `i32` 类型。

在 `calc` 函数入口处，首先为局部对象分配空间：

```
%values = alloca [8 x i32], align 16
%i = alloca i32, align 4
%result = alloca i32, align 4
%n = alloca i32, align 4
```

其中:

- values 对应 SysY 中的 int values[8];
- i 对应循环变量;
- result 保存累加结果;
- n 保存传入的函数参数。

参数 %n_arg 进入函数后，通过:

```
store i32 %n_arg, i32* %n, align 4
```

写入局部存储空间。
之后需要使用该变量时，再通过:

```
%n_val = load i32, i32* %n, align 4
```

取得实际值。
数组元素

$$\texttt{values}[i]$$

不能像高级语言一样直接使用下标访问，而需要显式计算元素地址。
首先将 32 位索引转换为 64 位:

```
%idx64 = sext i32 %i_cur to i64
```

然后使用 getelementptr:

```
%elem_ptr = getelementptr inbounds [8 x i32],
          [8 x i32]* %values,
          i64 0,
          i64 %idx64
```

这里第一个索引 0 表示仍然以当前数组对象为基础，第二个索引 %idx64 选择数组中的第 i 个元素。

因此，SysY 中一次简单的数组下标访问，在 LLVM IR 中被明确拆分为索引类型转换和地址计算。

5.3 控制流与整数运算

SysY 中的:

```
while (i < n)
```

在 LLVM IR 中由条件比较和基本块跳转实现。
循环条件部分为:

```
%i_val = load i32, i32* %i, align 4
%n_val = load i32, i32* %n, align 4
%loop_cmp = icmp slt i32 %i_val, %n_val
br i1 %loop_cmp, label %loop.body, label %loop.end
```

其中 `icmp slt` 执行有符号整数“小于”比较。
如果比较结果为真，则进入：

```
loop.body
```

否则跳转到：

```
loop.end
```

结束循环。
循环内部的奇偶判断：

```
if ((i + 1) % 2 == 0)
```

转换为：

```
%i_plus_1 = add nsw i32 %i_cur, 1
%mod2 = srem i32 %i_plus_1, 2
%is_even = icmp eq i32 %mod2, 0
br i1 %is_even, label %if.then, label %if.else
```

其中：

- `add` 对应整数加法；
- `srem` 对应有符号整数取模；
- `icmp eq` 判断余数是否等于 0；
- `br` 根据比较结果选择不同基本块。

偶数分支中的平方运算：

```
%square = mul nsw i32 %i_plus_1, %i_plus_1
```

使用 `mul` 实现。
奇数分支中的一元负号：

```
%negative = sub nsw i32 0, %i_plus_1
```

通过 $0 - x$ 转换为减法实现。
主函数中的整数除法与取模分别使用：

```
%q = sdiv i32 %n2, 2
%r = srem i32 %n2, 2
```

因此，SysY 中的基本算术运算均可以映射到对应的 LLVM IR 整数指令。

```
31
32  loop.body:
33    %i_cur = load i32, i32* %i, align 4
34    %i_plus_1 = add nsw i32 %i_cur, 1
35
36    %mod2 = srem i32 %i_plus_1, 2
37    %is_even = icmp eq i32 %mod2, 0
38
39    %idx64 = sext i32 %i_cur to i64
40
41    %elem_ptr = getelementptr inbounds [8 x i32],
42                [8 x i32]* %values,
43                i64 0,
44                i64 %idx64
45
46    br i1 %is_even, label %if.then, label %if.else
47
48  if.then:
49    %square = mul nsw i32 %i_plus_1, %i_plus_1
50    store i32 %square, i32* %elem_ptr, align 4
51    br label %if.end
52
53  if.else:
54    %negative = sub nsw i32 0, %i_plus_1
55    store i32 %negative, i32* %elem_ptr, align 4
56    br label %if.end
57
```

图 6: 数组寻址与条件控制流的 LLVM IR 关键实现

5.4 函数调用与 SysY 运行时接口

SysY 程序依赖课程运行时提供输入输出函数。

在 LLVM IR 中首先声明:

```
declare i32 @getint()
declare void @putint(i32)
declare void @putch(i32)
```

这些声明只给出函数类型，并没有在当前 `main.ll` 中定义函数体。具体实现将在链接阶段由 SysY 运行时库提供。

读取输入时使用:

```
%input = call i32 @getint()
```

返回值直接作为 SSA 值保存在 `%input` 中。

用户自定义函数调用:

```
%calc_result = call i32 @calc(i32 %n2)
```

表示将 `%n2` 作为参数传入 `calc`，并取得一个 `i32` 返回值。

最终输出使用:

```
call void @putint(i32 %sum3)
call void @putch(i32 10)
```

其中 `putch(10)` 输出换行字符。

与后续 RISC-V 汇编实现相比，LLVM IR 的函数调用仍然保留了较高层的函数和参数类型信息。此时无需显式指定参数寄存器或保存返回地址，这些更接近目标体系结构的细节由后端代码生成阶段继续处理。

5.5 编译、链接与执行验证

在实际验证手写 `main.ll` 时，当前 WSL 中用于该步骤的 Clang 工具链对统一 `ptr` 形式的 opaque pointer 语法未默认启用。因此，为兼容实际验证环境，本实验采用 typed pointer 写法，例如:

```
i32*
```

以及：

```
[8 x i32]*
```

而不能直接使用较新 LLVM IR 中统一的 `ptr` 写法。

完成 `main.ll` 后，首先由 Clang 将 LLVM IR 转换为本机目标代码，然后与 SysY x86 运行时库链接：

```
clang main.ll -L../lib -lsysy_x86 -o main
```

链接完成后，首先使用：

```
echo 5 | ./main
```

进行单组验证，程序输出：

```
17
```

与人工计算结果一致。

随后使用统一测试集：

```
for n in 0 1 4 5 8 10; do
   echo "Input: $n"
   echo "$n" | ./main
done
```

得到结果：

表 6: LLVM IR 实现测试结果

输入	实际参与计算的 n	预期结果	LLVM IR 输出
0	1	3	3
1	1	3	3
4	4	21	21
5	5	17	17
8	8	111	111
10	8	111	111

六组测试全部与预期值一致。

输入 0 和 10 分别验证了下界和上界修正；其余输入同时验证了基本块跳转、数组寻址、循环、整数运算以及函数调用。

因此，在当前测试范围内，可以确认手写 LLVM IR 与原 SysY 程序保持一致的可观察行为。

```
(base) fall@LAPTOP-M4C35AJ1:~/nku-compiler/prework1/part2/llvm$ for n i
n 0 1 4 5 8 10; do
    echo "Input: $n"
    echo "$n" | ./main
done
Input: 0
3
TOTAL: 0H-0M-0S-0us
Input: 1
3
TOTAL: 0H-0M-0S-0us
Input: 4
21
TOTAL: 0H-0M-0S-0us
Input: 5
17
TOTAL: 0H-0M-0S-0us
Input: 8
111
TOTAL: 0H-0M-0S-0us
Input: 10
111
TOTAL: 0H-0M-0S-0us
(base) fall@LAPTOP-M4C35AJ1:~/nku-compiler/prework1/part2/llvm$
```

图 7: LLVM IR 六组输入执行结果

6 RISC-V 汇编实现与验证

6.1 运行环境与总体实现

主要负责人：谭锦睿（2412771）

RISC-V 部分采用 RV64 汇编手工实现上述 SysY 程序。由于实验机本身为 x86-64 环境，因此程序使用 RISC-V 交叉工具链生成目标文件，再通过 qemu-riscv64 执行。

正式实现前,首先编写只调用 `putint` 和 `putch` 的最小汇编程序,并与 `libsysy_riscv.a` 链接。程序能够正确输出 123，说明交叉编译器、SysY RISC-V 运行时库以及 QEMU 已经能够形成完整执行链路。

```
tjr@tjry7000p:~/compiler-lab/prework1/part2/riscv$ qemu-riscv64 ./runtime_test
123
TOTAL: 0H-0M-0S-0us
```

图 8: SysY RISC-V 运行时库验证

随后实现完整的 `calc` 和 `main`。主要需要解决四个问题：局部变量如何保存、数组下标如何转换为实际地址、高级语言控制结构如何变成分支与跳转，以及函数参数和返回值如何在寄存器之间传递。

6.2 栈帧与数组寻址

`calc` 函数包含 `values[8]`、`i`、`result` 以及参数 `n`。一个 SysY `int` 占 4 字节，因此数组本身需要 32 字节。实现中统一为该函数分配 48 字节栈帧。

表 7: `calc` 函数主要栈空间布局

位置	内容
`0(sp)--28(sp)`	`values[0]--values[7]`
`32(sp)`	`i`
`36(sp)`	`result`
`40(sp)`	参数 `n`

48 字节本身是 16 的整数倍，因此调用过程中能够保持栈对齐。

对于数组元素

$$\texttt{values}[i],$$

由于每个元素宽度为 4 字节，因此其地址满足：

$$\mathrm{addr}(\texttt{values}[i]) = \mathrm{sp} + 4i.$$

汇编中可以先通过左移两位得到 $4i$：

$$i \ll 2 = 4i,$$

再与 `sp` 相加得到数组元素地址。

这个过程体现出高级语言中的数组下标访问，在机器层实际上需要转换为“基地址 + 偏移量”的地址计算。

6.3 控制流与整数运算

源程序中的：

$$\texttt{while}(i < n)$$

在汇编中不再存在一个独立的“循环”指令，而是转换为比较和跳转。

每轮循环开始时先比较 i 与 n。如果

$$i \geq n,$$

则跳转到循环结束标签；否则继续执行循环体。循环体结束后再无条件跳回条件判断位置。

同样，源程序中的：

$$(i + 1) \bmod 2 == 0$$

使用 `remw` 计算余数，再根据结果是否为 0 跳转到不同分支。因此，从目标代码角度看，`while`、`if` 和 `else` 最终都可以归结为基本块之间的控制流转移。

虽然目标体系结构为 RV64，但 SysY 中的 `int` 为 32 位整数。因此本程序主要使用：

`addw`、`addiw`、`subw`、`mulw`、`divw`、`remw`

等 32 位整数运算指令，使目标代码的整数语义与 SysY 保持一致。

6.4 函数调用与运行时库

RISC-V 调用约定规定整数参数和返回值优先通过 a0--a7 等参数寄存器进行传递。本程序调用：

$$\texttt{calc}(n)$$

之前，首先将参数 n 放入 a0；函数结束后，返回结果同样通过 a0 返回。

main 还需要调用：

$$\texttt{getint}, \quad \texttt{putint}, \quad \texttt{putch}.$$

这些函数由 SysY 运行时库提供。

由于执行 call 会覆盖返回地址寄存器 ra，因此 main 在函数入口处先将 ra 保存到栈中，结束前再恢复。

这一实现说明，高级语言中的一次函数调用，在目标代码层面实际上还需要处理参数位置、返回值、返回地址和栈空间恢复等问题。

6.5 汇编、链接与执行验证

首先将汇编文件生成可重定位目标文件：

```
riscv64-unknown-elf-gcc main.s -c -o main.o -w
```

然后与 SysY RISC-V 运行时库进行静态链接：

```
riscv64-unknown-elf-gcc main.o -o main \
  -L../lib -lsysy_riscv -static -mcmodel=medany \
  -Wl,--no-relax,-Ttext=0x90000000
```

其中 -L../lib 指定运行时库目录，-lsysy_riscv 链接课程提供的 RISC-V SysY 运行时库，-static 表示使用静态链接。

生成结果如图 9 所示。可以看到，main.o 为 RISC-V 可重定位目标文件，而最终的 main 已经是静态链接的 64 位 RISC-V ELF 可执行程序。

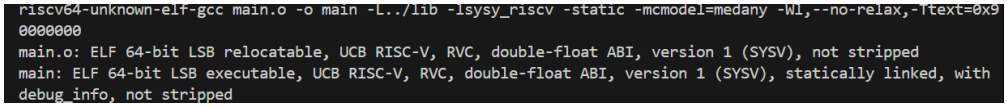


图 9: RISC-V 汇编与链接结果

验证阶段选取 0、1、4、5、8 和 10 六组输入。除了普通输入外，0 和 10 专门用于检查源程序中的上下界修正逻辑。

表 8: RISC-V 程序测试结果

输入	实际参与计算的 n	预期结果	RISC-V 输出	验证
0	1	3	3	正确
1	1	3	3	正确
4	4	21	21	正确
5	5	17	17	正确
8	8	111	111	正确
10	8	111	111	正确

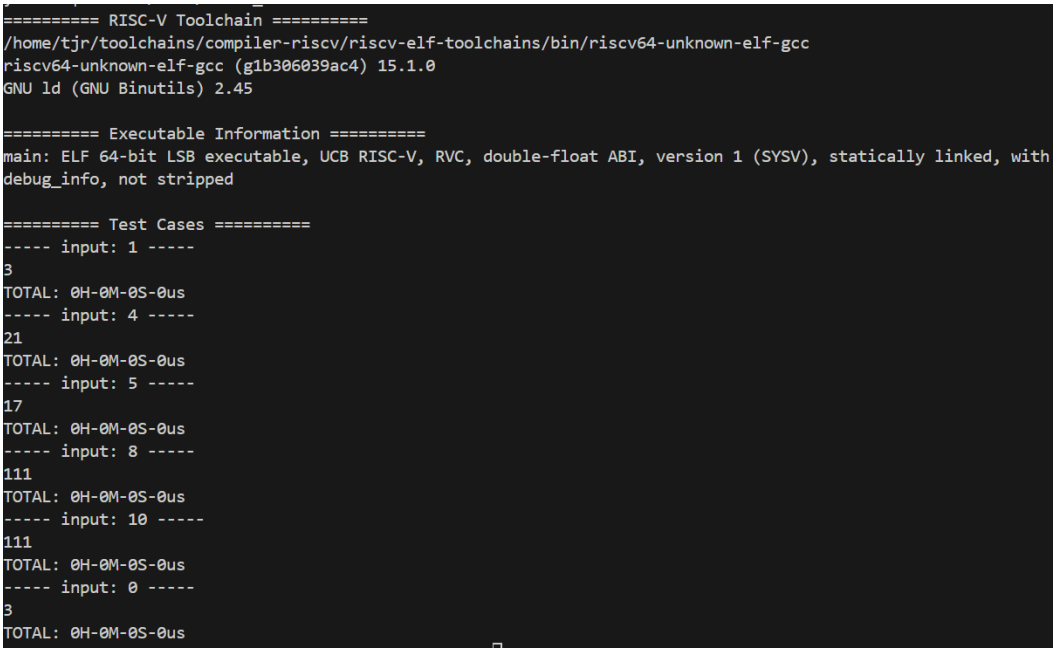


图 10: RISC-V 多组输入执行结果

六组测试全部得到预期结果。输入 0 被修正为 1，输入 10 被限制为 8，说明上下界条件分支正确；其余输入的结果同时验证了循环、数组地址计算、乘除取模、函数调用以及运行时输出等实现。

因此，可以确认手写 RISC-V 程序在这些测试输入下与原 SysY 程序保持一致的可观察行为。

7 LLVM IR 与 RISC-V 实现结果比较

为了验证两种低层实现是否保持相同的 SysY 程序语义，分别对 LLVM IR 和 RISC-V 汇编版本使用相同的六组输入进行测试。结果如表所示。

表 9: LLVM IR 与 RISC-V 执行结果比较

输入	预期结果	LLVM IR 输出	RISC-V 输出
0	3	3	3
1	3	3	3
4	21	21	21
5	17	17	17
8	111	111	111
10	111	111	111

六组输入下，LLVM IR 与 RISC-V 汇编实现均得到与预期值相同的结果。其中输入 0 和 10 还分别验证了程序的下界修正和上界限制逻辑。因此，在本实验覆盖的测试范围内，两种低层表示均保持了 SysY 源程序相同的可观察行为。

8 进阶探索：MLIR 与 AscendNPU IR

8.1 从 LLVM IR 到多层中间表示

基础实验中的 LLVM IR 提供了一种较统一的中间表示。C 程序被转换为 `alloca`、`load`、`store`、`icmp`、`br` 等较低层操作之后，源语言中的大部分高级结构已经被消除。这种设计有利于为不同前端和不同目标机器建立统一接口。

但面向 AI 加速器时，程序通常还包含张量、向量运算、专用存储空间和数据搬运等更丰富的语义。如果过早全部下降到统一低层指令，这些信息可能难以继续用于针对硬件特点的优化。

MLIR，即 Multi-Level Intermediate Representation，采用多层中间表示思想，通过 Dialect 描述不同抽象层次的操作。编译过程可以先保留较高层领域语义，再经过多个 Pass 逐步下降：

$$\text{High-level IR} \rightarrow \text{Domain Dialect} \rightarrow \text{Hardware Dialect} \rightarrow \text{Low-level IR}.$$

因此，lowering 不再只是“一次从高级语言变成低层 IR”，而可以理解为多个中间表示之间连续降低抽象层次的过程。

8.2 AscendNPU IR 官方 VecAdd 示例分析

根据 AscendNPU IR 官方快速入门中的 VecAdd 示例，其输入和输出均使用带有显式存储空间属性的 `memref` 表示。例如，全局内存中的数据使用：

```
memref<16xi16, #hivm.address_space<gm>>
```

而片上 Unified Buffer 中的临时数据使用：

```
memref<16xi16, #hivm.address_space<ub>>
```

这与普通 LLVM IR 中相对通用的内存指针存在明显区别：AscendNPU IR 在中间表示阶段已经显式保留了数据所在的硬件存储层次。

官方 VecAdd 的主要计算过程可以抽象为：

$$\text{GM} \xrightarrow{\texttt{hivm.hir.load}} \text{UB} \xrightarrow{\texttt{hivm.hir.vadd}} \text{UB} \xrightarrow{\texttt{hivm.hir.store}} \text{GM}.$$

其中，两路输入首先分别从 GM 搬运到 UB：

```
hivm.hir.load ...
```

随后在 UB 上执行向量加法：

```
hivm.hir.vadd ...
```

最后将结果重新写回 GM：

```
hivm.hir.store ...
```

因此，虽然源语义只是简单的

$$C = A + B,$$

但面向 NPU 的中间表示还需要显式表达数据位于何处、数据如何移动以及使用何种硬件操作完成计算。

官方示例使用如下命令进行编译：

```
bishengir-compile add.mlir -enable-hivm-compile -o kernel.o
```

最终生成可以在昇腾 NPU 上执行的设备侧算子二进制 `kernel.o`。

由于本实验环境没有配置 CANN 以及昇腾 NPU 设备，因此没有将该示例的上板执行结果作为本机实验结果，而是按照题目允许的方式，通过官方文档和 IR 示例对其中端表示进行分析。

8.3 多级 IR 与逐层 Lowering

AscendNPU IR 建立在 MLIR 基础上。与 LLVM IR 主要围绕一种核心中间表示展开不同，MLIR 允许不同 Dialect 在多个抽象层次上共存，使编译器能够在仍然保留高层语义时进行领域相关优化，再逐步降低到接近硬件的表示。

从 AscendNPU IR 的整体接口设计可以将这种过程理解为：

$$\text{Torch IR} \rightarrow \text{Linalg / HFusion IR} \rightarrow \text{HIVM IR} \rightarrow \text{NPU Device Binary.}$$

较高层 IR 主要描述框架算子和张量计算语义；Linalg/HFusion 层可以进一步进行融合、tiling 和调度；HIVM IR 则更加接近昇腾硬件，能够显式表示 GM、UB 等存储空间以及向量计算和数据搬运操作。

需要说明的是,官方快速入门中的 VecAdd 示例已经直接给出了较接近硬件层的 HIVM MLIR，因此示例本身并没有完整展示从 Torch IR 开始的每一个转换步骤。本实验结合官方的多级 IR 文档理解其整体 lowering 思路，而不将文档中未实际展示的中间 Pass 输出作为本机实验结果。

这一设计说明，多级 lowering 的意义并不只是把一种文本格式连续改写为另一种文本格式，而是在不同阶段有选择地保留适合当前优化任务的语义信息。较高层阶段保留张量和算子结构，较低层阶段则逐渐显式化存储层次、数据搬运和硬件指令。

8.4 与 LLVM IR 的比较

表 10: LLVM IR 与 MLIR/AscendNPU IR 的特点比较

比较项	LLVM IR	MLIR / AscendNPU IR
抽象层次	以相对统一的低层表示为核心	允许多个不同抽象层次的 Dialect 共存
扩展方式	围绕统一 IR 指令体系进行处理	可以面向领域和硬件定义专用 Dialect
硬件信息	通常在后端逐步体现	可以较早保留存储空间、数据搬运和向量操作
优化方式	主要围绕 LLVM IR 和目标后端	可以在不同 Dialect 层分别执行针对性优化

本实验使用普通 WSL2 环境，没有配置完整 CANN 和昇腾 NPU 设备，因此进阶部分主要采用官方示例阅读和 IR 静态分析的方式完成，没有将官方示例中的设备运行结果作为本机实验结果。

通过这一探索可以看出，现代 AI 编译器的中间端并不一定只依赖一种统一 IR，而可以在不同层次保留不同程度的语义信息，并通过渐进式 lowering 最终转换为目标硬件可以执行的代码。

9 总结

本实验围绕编译器从高级语言到目标程序的完整处理过程展开。第一部分通过 GCC 和 Clang/LLVM 的实际输出观察了预处理、词法分析、AST、LLVM IR、汇编、目标文件和链接等阶段，使课堂中较为抽象的编译流程与实际文件和指令建立了对应关系。

对 `-O0` 与 `-O2` 汇编代码的比较还表明，编译优化会重新组织寄存器使用、循环结构甚至函数调用方式，而不是简单地对源程序进行逐句翻译。优化的目标是在保持程序语义一致的前提下减少不必要的访存、分支和函数调用等开销。

第二部分首先共同设计统一的 SysY 测试程序和测试用例，并在此基础上分别完成 LLVM IR 与 RISC-V 两种低层表示。LLVM IR 部分涉及基本块划分、局部变量与数组表示、`getelementptr` 地址计算、条件和循环控制流、运行时函数声明及链接；RISC-V 部分进一步涉及局部变量的栈上布局、数组的基址加偏移寻址、RV64 环境中的 32 位整数运算以及函数调用约定。两种实现均使用 0、1、4、5、8、10 六组输入验证，并得到与 SysY 源程序一致的结果。进阶部分进一步将对中间表示的认识从 LLVM IR 扩展到 MLIR 与 AscendNPU IR。通过分析官方 VecAdd 示例可以看到，面向 NPU 等专用硬件时，中间表示不仅需要描述“执行什么计算”，还需要显式表达数据所在的存储空间、数据搬运过程以及相应的硬件计算操作。MLIR 通过 Dialect 和逐层 lowering，使编译器能够在不同抽象层次保留适合当前优化阶段的信息，最终逐步降低为面向目标硬件的低层表示。

通过本次实验，可以更加具体地认识到，编译器并不是简单的“源代码到机器码”转换工具，而是由前端分析、中间表示、优化、目标代码生成、汇编、链接以及运行时支持共同构成的完整系统。

参考文献

1. LLVM Project. *LLVM Language Reference Manual*. LLVM Documentation.
2. GCC Project. *Using the GNU Compiler Collection (GCC)*. GNU Project Documentation.
3. RISC-V International. *The RISC-V Instruction Set Manual*.
4. MLIR Project. *MLIR Documentation*. LLVM Project.
5. AscendNPU IR. *AscendNPU IR Documentation: Quick Start and VecAdd Example*. Huawei / AscendNPU IR Project.
6. 华为昇腾. *CANN* 编译器与毕昇编译器相关文档.